

Linux & git 보고서 작성

로봇 21기 예비단원 최윤서

1. 운영체제

1-1. 운영체제란?

- 메모리, CPU, 입출력 장치, 파일 스토리지 등의 리소스를 할당하여 컴퓨터의 하드웨어와 애플리케이션을 관리하는 소프트웨어 모음
- 작업 관리를 위해 맞춤형 시스템 소프트웨어가 필요했던 초기 컴퓨터에서 시작됨
배치 지향적 운영 체제에서 멀티태스킹 및 대화형 인터페이스를 지원하도록 발전

1-2. 운영체제의 발전

- Unix는 멀티태스킹, 이식성, 계층적 파일 시스템 개념 도입 -> 현대 운영 체제의 중요한 선례 마련함
- GPU 개발 -> 기업들이 GPU를 운영체제에 깊이 통합하기 시작

2. 리눅스의 역사와 발전

2-1. Unix에서 시작된 배경

- Unix: C언어 기반으로 출시된 운영 체제, 안정성이 가장 큰 장점 -> 기업들이 많이 이용
- Linux: 리눅스 기반의 x86 서버는 저렴한 가격, 벤더 종속성이 없는 기술 저변의 장점을 내세워 시장 점유율을 늘려감

2-2. GNU 프로젝트와의 관계

- GNU 프로젝트: 운영체제를 만들기 위한 도구들을 개발했던 프로젝트
GCC, Shell, 라이브러리 등이 포함됨.
사용자가 소스 코드를 자유롭게 확인, 수정, 배포 가능하도록 하자는 취지
하지만, 가장 핵심적인 부분인 커널이 없었음
- 핀란드 대학생 리누스 토르발스가 개인적으로 개발한 Linux라는 커널이 등장함
- GNU 프로젝트로 완성된 도구들과 Linux 커널 결합 -> 완전한 기능을 갖춘 운영체제인 Linux가 탄생함. 엄밀히 말하면 GNU/Linux 운영 체제임
- 결론적으로 GNU는 각각 운영체제의 다른 부분을 담당함

2-3. 오픈소스 운동과 라이선스 개념

- 오픈소스 라이선스란?
 - 보통 블로그나 타 정보를 이용하려면 저작권이 존재함.
 - 그러나 오히려 오픈소스 계에서는 사람들에게 마음대로 사용하라고 표시해놓은 것이 오픈소스 라이선스임.
- 오픈소스 운동: 소프트웨어를 누구나 자유롭게 사용하여 개발하고 연구할 수 있어야한다는 사상, 소스코드를 전세계에 배포하여 개발자들의 실력을 키우자는 취지
- 오픈소스 라이선스의 종류: MIT, BSD, GPL, Apache
 - MIT: - 가장 널리 알려진 라이선스. 매우 유연한 라이선스이고 존재하는 거의 모든 라이선스와 서로 호환된다.
 - 저작권 고지만 유지되면 소스 코드를 사용, 수정, 배포하는데 제한이 없다.
 - MIT 사용 조건: 이 소프트웨어를 누구라도 무상으로 제한없이 취급해도 좋으나, 저작권 표시 및 이 허가 표시를 소프트웨어의 모든 복제물 또는 중요한 부분에 기재할 것
 - 저자 또는 저작권자는 소프트웨어에 관해서 아무런 책임을 지지 않는다.
 - BSD: - BSD 운영체제를 포함한 BSD 프로젝트 소프트웨어에 사용됨.
 - MIT와의 차이점은 홍보/보증을 위한 목적으로 사용을 금지한다.
MIT 라이선스에 저작자의 이름이나 기여자의 이름을 사용하여 해당 소프트웨어의 승인이나 지원을 시사할 수 없다는 조건이 추가됨.
 - BSD 라이선스에서도 크게 3가지 라이선스 종류가 존재함.
 1. 2-clause BSD License: 저작권 고지문 및 라이선스 사본을 포함하여 소스코드를 사용, 수정, 배포할 수 있다. 소스 코드의 사용, 수정, 배포에 따른 책임이 제한됨.
 2. 3-clause BSD License: 위와 동일한 조건에 추가로, 저작자의 이름이나 기여자의 이름을 사용하여 해당 소프트웨어의 승인이나 지원 시사 불가능
 3. 4-clause BSD License: 위와 동일한 조건에 추가로, 소프트웨어를 사용하는 모든 광고 자료에 원 저작자의 이름을 포함해야함

3. 리눅스의 구조

3-1. 커널이란?

3-1-1. 커널의 기능

- 메모리 관리: 메모리가 어디서 무엇을 저장하는 데 얼마나 사용되는지 추적
- 프로세스 관리: 어느 프로세스가 CPU를 언제 얼마나 사용할지 결정
- 장치 드라이버: 하드웨어와 프로세스 사이에 중재자, 인터프리터 역할 수행
- 시스템 호출 및 보안: 프로세스의 서비스 요청 수신
- 올바르게 구현된 커널은 사용자가 볼 수 없음
- 커널은 하드웨어를 위해 분주히 일하는 개인 비서 느낌

3-1-2. 커널과 시스템 프로그램

- 운영체제는 커널과 시스템 프로그램으로 구분됨
- 커널은 운영체제의 핵심으로 디바이스 관리, 프로세스 관리, 메모리 관리 등 컴퓨터 자원을 관리
- 컴퓨터 자원을 관리하지 사용자와 직접적인 상호작용은 안함
- 따라서 사용자와 상호작용하기 위해 필요한 시스템 프로그램이 있어야됨

3-2. 배포판이란?

- 리눅스 커널 기반으로 각종 응용 프로그램, 패키지 관리자, GUI 등을 함께 모아놓은 종합 운영체제
- Ubuntu, Debian, CentOS, Red Hat Enterprise 등이 있음
- 사용자가 컴퓨터를 쉽게 설치 및 조작 가능하게하며 프로그램을 쉽게 다운받게함

3-3. 커널과 배포판의 관계

- 리눅스 커널들을 받아 기본 도구 세트를 추가한 것이 배포판
- 사람들이 보통 “리눅스를 쓴다” 라고하는건 리눅스 배포판을 쓴다는 소리(Ubuntu, Debian 등)

3-4. 리눅스 아키텍처 (커널 - 셸 - 응용프로그램 계층 구조)

3-4-1. 하드웨어 계층

- 시스템 운영에 필요한 모든 물리적 장치들이 포함됨
- CPU 기억, 그리고 입출력 장치 저장 드라이브, 네트워크 인터페이스 등이 포함됨
- 물리적 자원들과 직접 통신을 처리하여 메모리 할당과 데이터 처리 같은 작업 가능하게함

3-4-2. 커널 계층

- 리눅스 운영체의 핵심
- 하드웨어와 직접 상호작용하며 모든 중요한 시스템 자원을 관리함
- 핵심 기능은 3-1-1을 참고하시기 바랍니다.

3-4-3. 시스템 라이브러리

- 애플리케이션과 커널 간의 연결
- 커널과 상호작용 하는 데 사용하는 함수를 제공하는 데 중요한 역할
- 이 함수들을 커널 연산의 복잡성을 추상화하여 개발자가 커널과 직접 인터페이스하지 않고도 시스템 작업을 수행 가능하게함
- 가장 널리 사용되는 라이브러리는 GNU C 라이브러리(글리브크)

3-4-4. 시스템 유틸리티

- 시스템 관리 및 유지보수 작업을 돕는 프로그램
- 기본 유틸리티: 파일과 디렉터리를 관리하는 ls, cp, mv, rm 같은 명령어들
- 고급 유틸리티: 프로세스 모니터링, 방화벽 관리와 같은 복잡한 시스템 작업을 위한 ps, top, df 같은 도구들

3-4-5. 사용자 애플리케이션

- 사용자 공간에서 실행되며 웹 브라우저, 텍스트 편집기, 미디어 플레이어 등과 같은 사용자가 개발한 모든 맞춤형 프로그램이 포함됨

3-5. 파일시스템 구조

- /: 리눅스 파일 체제의 최상위 디렉토리, 모든 디렉토리의 시작점
- bin: 리눅스 기본 명령어 포함, 시스템 운영을 위한 기본 명령어 포함
- boot: 부팅에 핵심적인 커널 이미지와 부팅 정보 파일을 담음
- dev: 장치 파일들이 저장되어있는 디렉토리, 연결되어있는 장치 정보 확인가능
- etc: 시스템 환경 설정 파일이 있는 디렉토리, 네트워크 관련 설정 파일, 사용자 정보 및 암호정보, 파일 시스템 정보, 보안파일 등
- home: 리눅스 사용자의 홈 디렉토리가 만들어지는 디렉토리
- media: CD-ROM이나 USB같은 외부 장치 연결하는 디렉토리
- mnt: 파일 시스템을 임시로 연결하는 디렉토리
- root: 시스템 관리자의 홈 디렉토리, 일반 사용자는 접근 X
- sbin: bin 디렉토리와 유사 but, 오직 루트 유저만 실행할 수 있는 프로그램
- sys: 리눅스 커널 관련 정보가 있는 디렉토리
- tmp: 시스템 사용 중 발생한 임시 데이터 저장되는 디렉토리, 부팅 시 초기화
- usr: 기본 실행 파일과 라이브러리 파일, 헤더 파일 등의 파일이 저장되어있는 디렉토리, 대부분의 응용 프로그램과 파일이 저장됨
- var: 시스템 운영 중 발생한 데이터와 로그가 저장되는 디렉토리

3-6. 루트 디렉토리 & 홈 디렉토리

- 터미널 창을 실행 시켰을 때 \$ 앞에 있는 문자가 ~라면 홈 디렉토리이고 /는 루트 디렉토리임
- 루트 디렉토리: 3-5의 맨 첫번 째에 설명된 최상위 디렉토리이다.
- 홈 디렉토리: 여기서 말하는 홈 디렉토리는 루트 디렉토리 하위에 있는 home 디렉토리가 아니라 그 아래에 있는 사용자 home 디렉토리를 말함.
- 즉, 홈 디렉토리에서 상위로 두번 이동해야 루트 디렉토리가 나옴

4. 주요 배포판 소개 및 비교

4-1. Ubuntu

- Debian 기반에서 시작
- 편리한 설치와 사용법 때문에 데스크톱 환경에서 많이 사용됨
- 초기에 데스크톱 환경으로 가장 많이 사용하는 배포본으로 널리 알려졌고 이후 서버용으로 사용함
- 6개월 단위로 새 버전을 발표하며 2년 주기로 LTS 버전을 발표함
- pacemaker 활용한 구성이 가능하나, 문제 발생 시 기술 지원 없음
- DEB 기반 Apt를 활용한 패키지 의존성 관리

-

4-2. Debian

- 수많은 리눅스 배포판의 뿌리가 되는 운영체제
- apt 명령어를 통해 수만개의 소프트웨어를 자동으로 의존성 문제를 해결하며 쉽게 설치하고 업데이트할 수 있음
- 데비안 패키지는 검증이 매우 엄격하므로 충분히 검증된 프로그램만 안정 버전에 포함됨 -> 서버 운영에 최적화
- x86, ARM, MIPS 등 거의 모든 컴퓨터 프로세서 환경에서 작동함
- 완전 무료이며 자유 소프트웨어 가이드라인을 엄격히 준수함

4-3. CentOS

- Red Hat Enterprise Linux와 동일한 소스코드 기반으로 커뮤니티에서 새로 빌드해 제공함.
- Red Hat Enterprise Linux에 비해 일부 하드웨어 플랫폼, 드라이버, 특정 기능이 제외됨
- 국내 파트너사를 통해 1차 기술 지원을 받을 수 있으나 Red Hat으로 부터의 2,3차 기술 지원은 받을 수 없음
- 대부분의 주요 상용 애플리케이션은 CentOS를 인증하지않음
- RPM 기반 YUM/dnf를 활용한 패키지 의존성 관리

4-4. Fedora

- 최신 오픈소스 기술을 빠르게 도입하고 안정성과 개발자 편의성을 제공하는 운영체제
- 최신 리눅스 커널과 GNOME, KDE 등의 최신 데스크톱 환경을 빠르게 반영하여 실기술을 먼저 경험할 수 있음
- 빠르고 효율적인 DNF 패키지 관리자 사용 -> 소프트웨어 쉽게 설치, 업데이트 및 제거 가능
- 기업용 리눅스인 Red Hat Enterprise Linux의 업스트림 역할을 하므로 상용 리눅스 환경을 미리 테스트하기 좋음

4-5. Red Hat

- 미국 기반의 Linux 대표 기업
- 안정성을 최우선 -> 신기술/신기능 적용을 보수적으로 함
- 이중화 클러스터 포함됨
- PRM 기반 YUM/dnf를 활용한 패키지 의존성 관리

5. Window 운영체제와 비교

5-1. 비용적 측면

- window: 유료이며 고가임 / 기본 애플리케이션을 이용할 경우 추가 비용 발생
- Linux: 무료 라이선스로 사용 제한 없음 / 구축하고자하는 모든 시스템에서 추가 비용 없이 설치 가능

5-2. 안정성 및 신뢰도

- Linux: 안정성과 신뢰도를 높이는 운영체제로 인정 받음 / 개발 업체나 개발자들로부터 수시로 업데이트 되고있음
- 그러나 Linux는 디스크 입출력이 비동기화 방식이므로 시스템 충돌, 전원 문제 등이 발생할 경우 파일 시스템이 깨질 수 있는 우려도 있음

5-3. 성능

- window: 윈도우 개발언어로 개발된 서비스 페이지의 경우 윈도우OS가 설치된 서버에서 최고의 성능을 발휘
- Linux: 대부분 응용프로그램에 대해 좋은 성능을 발휘하며 낮은 성능의 서버로 고성능을 낼 수 있음

5-4. 보안성

- Linux: 다중 사용자 체제이므로 관리자 권한으로 로그인하지 않으면 모든 사용자는 보호 모드에서 작동하므로 윈도우에 비해 바이러스가 매우 적고 보안성이 높음
- Window: 시스템 버그나 보안 취약점 발견 시 패치가 나오는데 상당한 시일이 걸리는 게 사실임

5-5. 응용프로그램

- Linux: 다양한 무료 오픈 소스 프로그램이 존재 / 직접 소스 수정 및 제작하여 배포 가능
- Window: 무료 프로그램의 수는 적음 / 소스코드 없이 제공되기 때문에 사용자에게 의해 수정 혹은 개선이 불가능

5-6. 상용 애플리케이션

- Window: 다른 어떤 운영체제에 비해 많은 애플리케이션을 보유하고있다는 것이 최대 장점 / WMA 스트리밍 서비스 같이 ASP로 개발된 페이지를 구동해야 하는 경우가 대표적임
- Linux: 리눅스도 지원하는 상용 애플리케이션들은 점차 늘고있는 추세지만 아직 Window보단 적음

5-7. 운영관리 측면

- Window: 운영 관리적인 측면에서는 텍스트 입력 방식의 리눅스보단 GUI기반의 Window가 더 편리함.
- Linux: 단, 기술 지원 측면에서는 리눅스가 전문적이고 신속하게 이뤄짐

6. Git

6-1. Git이란?

- 소스 코드 변경 이력을 기록하고 관리하는 분산형 버전 관리 시스템
- 파일 내 변화를 시간의 흐름에 따라 기록 후 필요한 상황에서 그 파일을 꺼내올 수 있는 시스템

6-2. 왜 Git을 배워야하는가?

6-2-1. 팀과의 협력

- 다양한 프로젝트에서 github는 여러 사람이 같은 코드베이스에서 충돌없이 작업할 수 있게해줌
- 분기, 풀 리퀘스트, 코드 리뷰 같은 기능 덕분에 모두가 독립적으로 작업하면서도 프로젝트 안정성을 유지할 수 있음

6-2-2. 포트폴리오 구축

- Github 프로파일은 프로젝트의 실시간 포트폴리오임
- 인턴십, 졸업생 지원, 프리랜서 지원 시 채용 담당자들이 종종 Github를 확인해 본인의 역량을 평가함

7. Git 기본 개념

7-1. 로컬 저장소 VS 원격 저장소

7-1-1. 로컬 저장소

- 내 PC에 파일이 저장되는 개인 전용 저장소
- 이 저장소로 작업할 때는 네트워크 필요 X

7-1-2. 원격 저장소

- 파일이 전용 서버에서 관리됨
- 서버에 저장하기 위해서는 네트워크 필요
- 팀원과 함께 코드 변경 사항을 풀링하고 푸시하는 협업은 오직 원격 저장소에서만 가능

7-1-3. git의 로컬 저장소와 원격 저장소 연결하기

- git init -> 명령어 입력 후 디렉토리 내부에 .git이 생겼으면 git 저장소가 되었다는 것을 의미
- git add . -> git commit -m "success" -> 임의로 문서 하나를 만든 뒤 커밋
- git remote add origin 원격 저장소 주소 -> 로컬과 원격 저장소가 연결됨
- git push -u origin main -> 지역 저장소의 브랜치를 origin 으로 push 함

7-2. 브랜치(branch)

7-2-1. 개념

- 여러 사람이 함께 작업하다보면 서로 다른 버전의 코드가 만들어질 수 밖에 없음
- 이 때 여러 개발자들이 동시에 다양한 작업을 할 수 있게 만들어주는 기능이 Branch임
- 각자의 branch 내에서 마음대로 소스코드를 변경할 수 있고 이렇게 분리된 작업 영역에서 변경된 내용은 나중에 원래 버전과 비교해서 하나의 새로운 버전을 만들 수 있음.

7-2-2. 장점

- 매우 가벼움
- branch를 새로 만들고 branch 사이를 이동할 수 있음
- 다른 버전 관리 시스템과는 달리 Git은 branch를 만들어 작업하고 나중에 Merge하는 방법을 권장함

8. 기본 사용법(핵심 명령어)

8-1. Setup & Config

- git init: 새 레포지토리를 만들
- git remote: 추가, 이름 변경 등 기존에 존재하는 리포지토리를 변경함
- git clone: 기존에 존재하는 리포지토리의 모든 history를 복사함
- git config: 이름, email, 편집자 등을 설정할 수 있는 명령어

8-2. 변경 사항 기록

- git add [파일명]: 변경된 파일을 Staging Area 에 올림
- git commit -m "메시지" : 스테이징된 변경사항을 로컬저장소에 영구적으로 기록

8-3. 상태/이력 확인

- git status: 현재 변경된 파일과 상태 확인
- git commit -am "메시지": 추적된 파일의 수정사항을 add와 동시에 커밋
- git log: 저장소에 기록된 커밋 이력을 확인

8-4. 원격 저장소 연결

- git remote add origin <원격저장소 URL> : 새로 만든 저장소와 연결하기
- git remote -v: 연결된 저장소 확인하기
- git remote remove <이름>: 저장소와 연결 해제하기

9. 실무 활용

9-1. .gitignore

9-1-1. .gitignore에 포함되는 정보

- 용량이 커서 제외되어야 할 파일 혹은 디렉토리 경로
- 보안적인 문제에서 걸려 제외되어야 할 파일 혹은 디렉토리 경로
- 불필요하다고 판단되어 제외되어야 할 파일 혹은 디렉토리 경로

9-1-2. 사용법

- 파일명으로 제외하는 법: [ignoreFileName.js](#)
- 특정 파일만 제외하는 법: [src/ignoreFileName.js](#)
- 특정 디렉토리 기준 이하 파일들 제외하는 법: [node_module/](#)
- 특정 디렉토리 하위의 특정 확장자 제외하는 법: [src/*.txt](#)
- 특정 확장자 제외하기: [.txt](#)

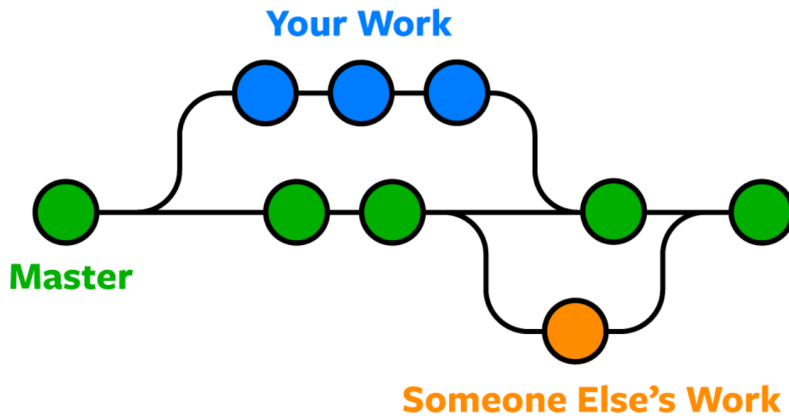
9-2. 커밋 메시지 작성 컨벤션

- feat: 새로운 기능 추가
- fix: 버그 수정
- refactor: 기능 변경없는 코드 구조 개선
- perf: 성능 개선
- docs: 문서 변경
- test: 테스트 추가/수정
- build: 빌드/의존성 관련
- ci: CI/CD 관련(CI/CD 파일: 개발의 자동화 과정을 정의해 둔 설정 파일)
- chore: 기타 작업
- style: 포매팅 등 코드 스타일
- revert: 이전 커밋 되돌리기

9-3. git rebase VS git merge 차이

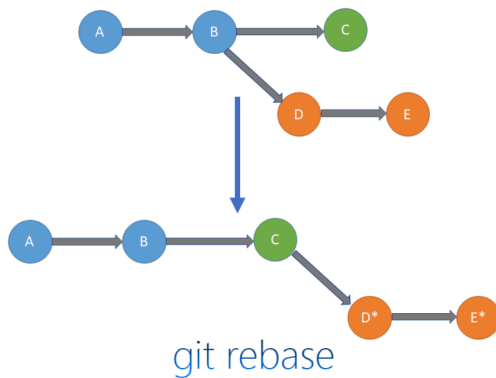
9-3-1. Merge

- branch를 통합하는 것



9-3-2. Rebase

- branch의 base 자체를 옮기는 것



- B 지점을 base로 가진 branch가 D,E 커밋 진행
- C 지점으로 base를 이동하기 위해 branch에서 C 지점으로 rebase함
- C 지점으로 rebase되면 기존 D,E 커밋을 새롭게 정렬되어 C 지점 이후로 변경됨